

WORKSHEET 4 – IMAGE PROCESSING (PERTEMUAN 11)

Nama: Cidesta Mentari Marintan Simbolon

NIM: 122400096

Kelas: Pengolahan Citra

Repository: https://github.com/cidesta122400096-coder/Tugas_Pengolahan_Citra_Cidesta_Mentari_122400096.git

PENDAHULUAN

Pada tugas ini dilakukan pengolahan citra digital menggunakan Python dengan bantuan library OpenCV. Pengolahan citra bertujuan untuk memahami bagaimana sebuah gambar dapat diproses secara komputasional, di mana citra direpresentasikan sebagai data matriks yang dapat diolah untuk menghasilkan informasi visual tertentu.

Dalam praktikum ini dilakukan beberapa tahapan dasar, yaitu pembacaan citra, konversi ruang warna dari BGR ke RGB, transformasi ke grayscale untuk menyederhanakan informasi warna, penerapan Gaussian Blur untuk mengurangi noise, serta deteksi tepi (edge detection) menggunakan metode Canny untuk mengekstraksi batas objek pada citra.

Tujuan

Tujuan dari worksheet ini adalah untuk memahami dasar-dasar pengolahan citra digital menggunakan Python dan library OpenCV. Selain itu, praktikum ini bertujuan untuk mempelajari bagaimana sebuah citra dapat diproses secara komputasional melalui beberapa tahapan seperti konversi warna, transformasi grayscale, peredaman noise menggunakan Gaussian Blur, serta deteksi tepi (edge detection) menggunakan metode Canny.

```
In [1]: import cv2
import matplotlib.pyplot as plt
```

```
In [5]: import cv2
import matplotlib.pyplot as plt

# =====
# LOAD IMAGE (3 GAMBAR)
# =====
img_object = cv2.imread("assets_ws4/object.jpg")
img_selfie = cv2.imread("assets_ws4/selfie.jpg")
img_tekstur = cv2.imread("assets_ws4/tekstur.jpg")
```

```

# =====
# CEK KALAU GAMBAR TIDAK TERBACA
# =====
if img_object is None:
    print("object.jpg tidak ditemukan")
if img_selfie is None:
    print("selfie.jpg tidak ditemukan")
if img_tekstur is None:
    print("tekstur.jpg tidak ditemukan")

# =====
# CONVERT BGR -> RGB
# =====
object_rgb = cv2.cvtColor(img_object, cv2.COLOR_BGR2RGB)
selfie_rgb = cv2.cvtColor(img_selfie, cv2.COLOR_BGR2RGB)
tekstur_rgb = cv2.cvtColor(img_tekstur, cv2.COLOR_BGR2RGB)

# =====
# TAMPILKAN 3 GAMBAR
# =====
fig, axs = plt.subplots(1, 3, figsize=(12, 5))

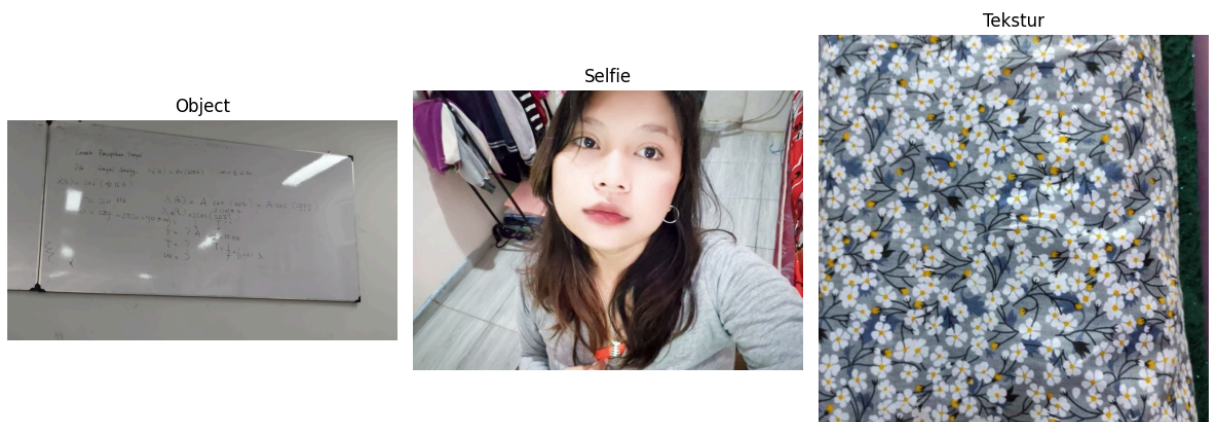
axs[0].imshow(object_rgb)
axs[0].set_title("Object")
axs[0].axis("off")

axs[1].imshow(selfie_rgb)
axs[1].set_title("Selfie")
axs[1].axis("off")

axs[2].imshow(tekstur_rgb)
axs[2].set_title("Tekstur")
axs[2].axis("off")

plt.tight_layout()
plt.show()

```



```

In [6]: gray_object = cv2.cvtColor(img_object, cv2.COLOR_BGR2GRAY)
        gray_selfie = cv2.cvtColor(img_selfie, cv2.COLOR_BGR2GRAY)
        gray_tekstur = cv2.cvtColor(img_tekstur, cv2.COLOR_BGR2GRAY)

```

```
fig, axs = plt.subplots(1, 3, figsize=(12, 4))

axs[0].imshow(gray_object, cmap='gray')
axs[0].set_title("Gray Object")
axs[0].axis("off")

axs[1].imshow(gray_selfie, cmap='gray')
axs[1].set_title("Gray Selfie")
axs[1].axis("off")

axs[2].imshow(gray_tekstur, cmap='gray')
axs[2].set_title("Gray Tekstur")
axs[2].axis("off")

plt.tight_layout()
plt.show()
```



```
In [7]: blur_object = cv2.GaussianBlur(img_object, (5,5), 0)
blur_selfie = cv2.GaussianBlur(img_selfie, (5,5), 0)
blur_tekstur = cv2.GaussianBlur(img_tekstur, (5,5), 0)

blur_object = cv2.cvtColor(blur_object, cv2.COLOR_BGR2RGB)
blur_selfie = cv2.cvtColor(blur_selfie, cv2.COLOR_BGR2RGB)
blur_tekstur = cv2.cvtColor(blur_tekstur, cv2.COLOR_BGR2RGB)

fig, axs = plt.subplots(1, 3, figsize=(12, 4))

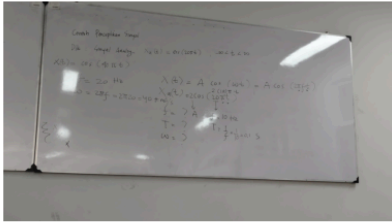
axs[0].imshow(blur_object)
axs[0].set_title("Blur Object")
axs[0].axis("off")

axs[1].imshow(blur_selfie)
axs[1].set_title("Blur Selfie")
axs[1].axis("off")

axs[2].imshow(blur_tekstur)
axs[2].set_title("Blur Tekstur")
axs[2].axis("off")

plt.tight_layout()
plt.show()
```

Blur Object



Blur Selfie



Blur Tekstur



```
In [8]: edge_object = cv2.Canny(img_object, 100, 200)
edge_selfie = cv2.Canny(img_selfie, 100, 200)
edge_tekstur = cv2.Canny(img_tekstur, 100, 200)

fig, axs = plt.subplots(1, 3, figsize=(12, 4))

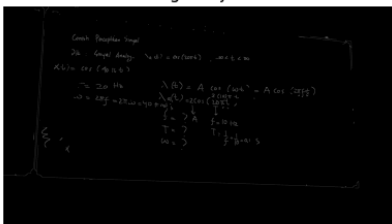
axs[0].imshow(edge_object, cmap='gray')
axs[0].set_title("Edge Object")
axs[0].axis("off")

axs[1].imshow(edge_selfie, cmap='gray')
axs[1].set_title("Edge Selfie")
axs[1].axis("off")

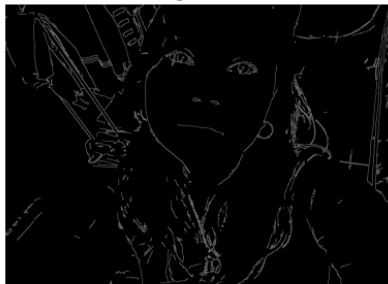
axs[2].imshow(edge_tekstur, cmap='gray')
axs[2].set_title("Edge Tekstur")
axs[2].axis("off")

plt.tight_layout()
plt.show()
```

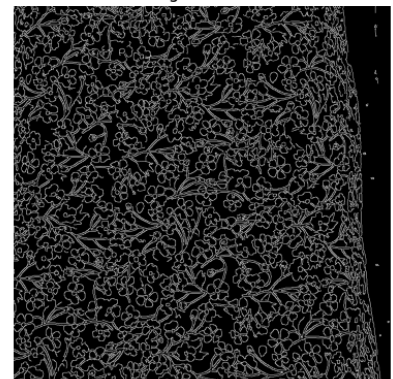
Edge Object



Edge Selfie



Edge Tekstur



Analisis

Berdasarkan hasil pengolahan citra yang dilakukan, setiap proses menghasilkan perubahan visual yang berbeda pada gambar. Pada tahap grayscale, citra diubah menjadi skala abu-abu sehingga informasi warna dihilangkan dan hanya menyisakan intensitas cahaya. Hal ini mempermudah analisis struktur gambar tanpa dipengaruhi warna.

Selanjutnya, pada proses Gaussian Blur, citra mengalami penghalusan dengan mengurangi noise atau gangguan kecil pada gambar. Hasil ini membuat detail gambar menjadi lebih lembut dan kurang tajam. Sementara itu, pada proses edge detection menggunakan metode Canny, diperoleh hasil berupa garis tepi objek yang menonjolkan bentuk dan struktur utama dari gambar. Proses ini berguna untuk mengidentifikasi batas objek dalam citra secara lebih jelas.

Kesimpulan

Pada worksheet4 ini telah dilakukan pengolahan citra digital menggunakan Python dengan library OpenCV. Proses yang dilakukan meliputi pembacaan citra, konversi warna, grayscale, Gaussian Blur, dan edge detection.

Dari hasil percobaan dapat disimpulkan bahwa setiap teknik pengolahan citra memiliki fungsi yang berbeda, seperti grayscale untuk menyederhanakan informasi warna, blur untuk mengurangi noise, dan edge detection untuk mengekstraksi tepi objek. Dengan demikian, pengolahan citra dapat digunakan untuk membantu analisis visual secara lebih efektif.

Lampiran Penggunaan AI (Pertemuan 11)

Dalam pengerjaan pengolahan citra image processing pertemuan 11, penulis menggunakan bantuan AI (ChatGPT) sebagai alat pendukung dalam memahami dan memperbaiki kode OpenCV, termasuk proses grayscale, Gaussian Blur, dan edge detection.

Link percakapan AI: <https://chatgpt.com/share/6a13cb5b-daf0-83ec-a4c0-9cc673aec8c1>